

Chapter 2: JavaScript and Client-Side Programming

By Er. Ukesh Thapa

Syllabus

2.1 JavaScript essentials

2.1.1 Data types, variables, control structures, functions

2.1.2 Dom manipulation, events, and form validation

2.1.3 Local storage and session storage

2.1.4 GUI interactions

2.1.5 JavaScript library: jQuery

2.2 Modern JavaScript (ES6+)

2.2.1 Arrow functions, destructuring, spread/rest operators

2.2.2 Callbacks, promises, async/await

2.2.3 Modules and imports

Syllabus

2.3 Client-side applications

2.3.1 Client-side web application development using React JS library

2.3.2 Traditional multi-page apps vs. single-page apps (SPAs)

2.3.3 Component-based UI and props

2.3.4 State management and data flow

2.3.5 Client-side routing and navigation without reloads

2.3.6 Fetch API: Fetching, displaying data, handling errors and loading states

2.3.7 Comparison of modern JavaScript libraries: React, Angular, Vue

Hours: 12

Marks Distribution: 16

Javascript (JS)

- ❖ Also Called as DHTML (Dynamic HTML)
- ❖ It was created in 10 days in 1995. It was named "JavaScript" purely as a marketing tactic because "Java" was popular then.
- ❖ The Browser Wars & The Birth of ECMAScript (1997):
 - The Solution: They went to a standards body called ECMA (European Computer Manufacturers Association) to write a rulebook.
- ❖ Major Updates: 2009 (ES5), 2015 (ES6 / ES2015) {The Big Bang}
- ❖ Now: Yearly updates.
 - Latest: ECMAScript 2024 (ES15)

"HTML is the skeleton. CSS is the skin. JavaScript is the brain."

Formal Definition JS

- ❖ "JavaScript is a high-level, interpreted, client-side scripting language used to create interactive and dynamic content on web pages."
- ❖ Breakdown of Keywords:
 - High-level: Easy for humans to read (close to English), handles memory automatically.
 - Interpreted: The browser reads the code line-by-line and executes it (no complex compiling step like C++).
 - Client-side: It runs on the user's computer (in the browser), not on the server.
 - Dynamic: It can change the webpage content after the page has loaded without refreshing.

JavaScript Variables and Constants

- ❖ A JavaScript variable is a container for storing data.
 - Two type of Variable
 - Var: Variables created with var are function-scoped
 - In the older version of JS we use var
 - example : `var variableName = value`
 - Let: Variables created with let are blocked-scoped
 - ES6 version of JS we use let
 - example : `let variableName = value`
- ❖ **Constants:**
 - A constant is a type of variable whose value cannot be changed.
 - Example: `const variableName = value`

Display with JS

❖ Alert:

- A built-in function that displays a pop-up dialog box.
- Syntax:
 - `alert("Your message goes here");`

❖ console.log():

- Displays messages or variables in the browser's console.
- Syntax:
 - `console.log("Message in console")`

JS: Data type

- ❖ There are two type of data type:
 - Primitive Data Types: They can hold a single simple value. String, Number, BigInt, Boolean, undefined, null, and Symbol are primitive data types.
 - Non-Primitive Data Types: They can hold multiple values. Objects are non-primitive data types.

Data Type	Description
String	Textual data. <code>'hello', "hello world!", etc.</code>
Number	An integer or a floating-point number. <code>3, 3.234, 3e-2, etc.</code>
BigInt	An integer with arbitrary precision. <code>900719925124740999n, 1n, etc.</code>
Boolean	Any of two values: true or false.
undefined	A data type whose variable is not initialized. <code>let a;</code>
null	Denotes a null value. <code>let a = null;</code>
Symbol	A data type whose instances are unique and immutable. <code>let value=Symbol('hello');</code>
Object	Key-value pairs of collection of data. <code>let student = {name: "John"};</code>

JavaScript String

A string represents textual data. It contains a sequence of characters.

In JavaScript, strings are surrounded by quotes:

- Single quotes: 'Hello'
- Double quotes: "Hello"
- Backticks: `Hello`

JavaScript Number

The number type represents numeric values (both integers and floating-point numbers).

- Integers - Numeric values without any decimal parts. Example: 3, -74, etc.
- Floating-Point - Numeric values with decimal parts. Example: 3.15, -1.3, etc.

JavaScript Symbol

A Symbol is a unique and primitive value. This data type was introduced in ES6.

// two symbols with the same description

```
let value1 = Symbol("programiz");
```

```
let value2 = Symbol("programiz");
```

```
console.log(value1 === value2); // false
```

JavaScript Operator

- ❖ Here is a list of different JavaScript operators you will learn in this tutorial:
 - Arithmetic Operators
 - Assignment Operators
 - Comparison Operators
 - Logical Operators
 - Bitwise Operators
 - String Operators
 - Miscellaneous Operators

JavaScript Arithmetic Operators

Operator	Name	Example
<code>+</code>	Addition	<code>3 + 4 // 7</code>
<code>-</code>	Subtraction	<code>5 - 3 // 2</code>
<code>*</code>	Multiplication	<code>2 * 3 // 6</code>
<code>/</code>	Division	<code>4 / 2 // 2</code>
<code>%</code>	Remainder	<code>5 % 2 // 1</code>
<code>++</code>	Increment (increments by 1)	<code>++5</code> or <code>5++ // 6</code>
<code>--</code>	Decrement (decrements by 1)	<code>--4</code> or <code>4-- // 3</code>
<code>**</code>	Exponentiation (Power)	<code>4 ** 2 // 16</code>

JavaScript Assignment Operators

Operator	Name	Example
=	Assignment Operator	<code>a = 7;</code>
+=	Addition Assignment	<code>a += 5; // a = a + 5</code>
-=	Subtraction Assignment	<code>a -= 2; // a = a - 2</code>
*=	Multiplication Assignment	<code>a *= 3; // a = a * 3</code>
/=	Division Assignment	<code>a /= 2; // a = a / 2</code>
%=	Remainder Assignment	<code>a %= 2; // a = a % 2</code>
=	Exponentiation Assignment	<code>a **= 2; // a = a2</code>

JavaScript Comparison Operators

Operator	Meaning	Example
<code>==</code>	Equal to	<code>3 == 5</code> gives us <code>false</code>
<code>!=</code>	Not equal to	<code>3 != 4</code> gives us <code>true</code>
<code>></code>	Greater than	<code>4 > 4</code> gives us <code>false</code>
<code><</code>	Less than	<code>3 < 3</code> gives us <code>false</code>
<code>>=</code>	Greater than or equal to	<code>4 >= 4</code> gives us <code>true</code>
<code><=</code>	Less than or equal to	<code>3 <= 3</code> gives us <code>true</code>
<code>===</code>	Strictly equal to	<code>3 === "3"</code> gives us <code>false</code>
<code>!==</code>	Strictly not equal to	<code>3 !== "3"</code> gives us <code>true</code>

JavaScript Logical Operators

Operator	Syntax	Description
<code>&&</code> (Logical AND)	<code>expression1 && expression2</code>	<code>true</code> only if both <code>expression1</code> and <code>expression2</code> are <code>true</code>
<code> </code> (Logical OR)	<code>expression1 expression2</code>	<code>true</code> if either <code>expression1</code> or <code>expression2</code> is <code>true</code>
<code>!</code> (Logical NOT)	<code>!expression</code>	<code>false</code> if <code>expression</code> is <code>true</code> and vice versa

JavaScript Comments

Types of JavaScript Comments

In JavaScript, there are two ways to add comments to code:

- ❖ `//` - Single-Line Comments
- ❖ `/* */` - Multiline Comments

JavaScript Type Conversion

There are two types of type conversion in JavaScript:

- ❖ Implicit Conversion - Automatic type conversion.
 - Example: `let result = "3" + 2;` // output is string
 - Example: `let result = "3" + true;` // output is string
 - Example: `let result = "3" + null;` // output is string
- ❖ Explicit Conversion - Manual type conversion.
 - `Number("5")`
 - `String(true)`
 - `Boolean(0)`
- ❖ Display data type: **`typeof(variable)`**

`document.body.style.background = "blue"`

JS: Condition

- ❖ If....else condition

```
if(condition){
```

```
  \\expression
```

```
}
```

```
else{
```

```
  \\expression
```

```
}
```

- ❖ If....else condition

```
if(condition){
```

```
  \\expression
```

```
}
```

```
else if (condition){
```

```
  \\expression
```

```
}
```

```
else{
```

```
  \\expression
```

```
}
```

JS: LOOP

For loop

```
for (initialExpression; condition; updateExpression) {  
    // for loop body  
}
```

Example:

```
1. for (let i = 0; i < 3; i++) {  
    console.log("Hello, world!");  
}
```

```
1. const fruits = ["apple", "banana", "cherry"];  
  
    for (let i = 0; i < fruits.length; i++) {  
        console.log(fruits[i]);  
    }
```

For While

```
while (condition) {  
    // body of loop  
}
```

For do while

```
do {  
    // body of loop  
} while(condition);
```

JS: Break and Continue

Break: The break statement terminates the loop immediately when it's encountered.

Keyword: **break**

Continue: The continue statement skips the current iteration of the loop and proceeds to the next iteration.

Keyword: **continue;**

switch...case Statement

```
switch (expression) {  
    case value1:  
        // code block to be executed  
  
        break;  
    case value2:  
        // code block to be executed  
  
        break;  
    default:  
        // code block to be executed  
  
}
```

```
let trafficLight = "green";  
let message = ""  
  
switch (trafficLight) {  
    case "red":  
        message = "Stop immediately.";  
        break;  
    case "yellow":  
        message = "Prepare to stop.";  
        break;  
    case "green":  
        message = "Proceed or continue driving.";  
        break;  
    default:  
        message = "Invalid traffic light color.";  
}  
  
console.log(message)  
  
// Output: Proceed or continue driving.
```

JavaScript Function

A function is an independent block of code that performs a specific task, while a function expression is a way to store functions in variables.

Syntax:

```
function function_name(name) {  
    // code  
}
```

- Pass arguments
- Return type

JS: Function Expressions

In JavaScript, a function expression is a way to store functions in variables. For example,

// store a function in the square variable

```
let square = function(num) {
```

```
    return num * num;
```

```
};
```

```
console.log(square(5));
```

// Output: 25

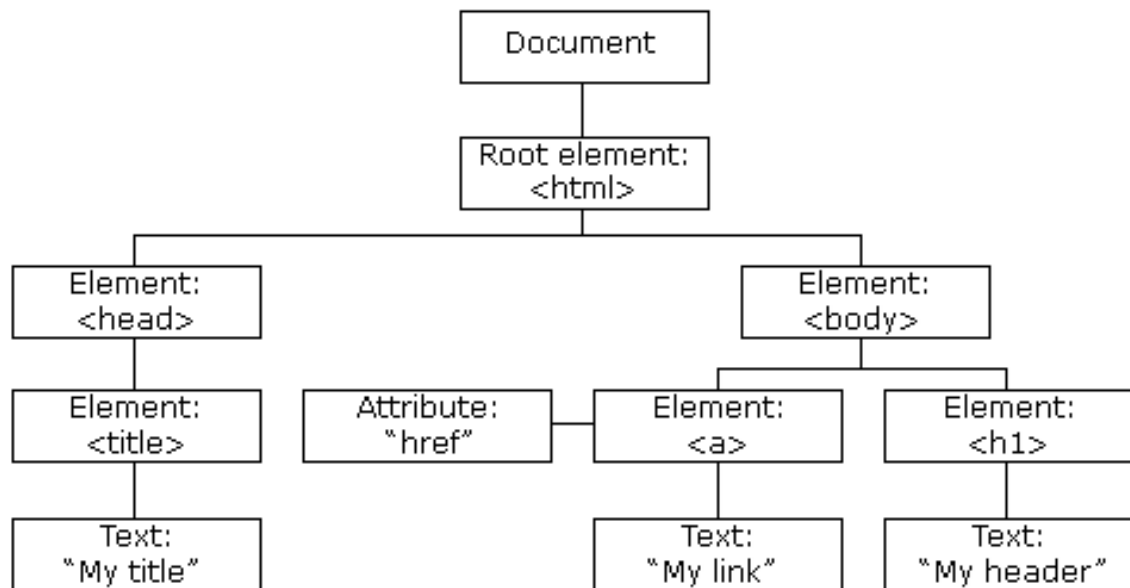
JavaScript Hoisting

In JavaScript, hoisting is a behavior in which a function or a variable can be used before declaration.

There are generally two types of hoistings in JavaScript:

- **Variable Hoisting**
 - In JavaScript, the behavior of hoisting varies depending on whether a variable is declared using `var`, `let`, or `const`.
- **Function Hoisting**
 - In JavaScript, function hoisting allows us to call the function before its declaration.

Document Object Model (DOM)



Document Object Model (DOM)

- ❖ JavaScript gets all the power it needs to create dynamic HTML:
 - JavaScript can change all the HTML elements in the page
 - JavaScript can change all the HTML attributes in the page
 - JavaScript can change all the CSS styles in the page
 - JavaScript can remove existing HTML elements and attributes
 - JavaScript can add new HTML elements and attributes
 - JavaScript can react to all existing HTML events in the page
 - JavaScript can create new HTML events in the page

DOM: Update UI

This is exactly what professional developers do to make a page "Dynamic."

❖ **Changing Content (Text & HTML)**

- This is the most common update. You replace the words on the screen.
- `.innerText`: Changes the visible text inside an element. (Safe & Fast).
- Code: `scoreDisplay.innerText = "Score: 100";`

❖ **Changing Visuals (Styles)**

- You can modify CSS directly from JavaScript to give immediate feedback (like turning an error message red).
- Example: `btn.style.backgroundColor = "green";`

❖ **Changing Attributes (Properties)**

- This changes the functionality or source of an element, not just the text.
- Example: `serImage.src = "avatar.png";`

❖ **Visibility (Show/Hide)**

- Making elements appear or disappear based on user logic (like closing a modal).
- `.style.display`: The standard way to hide ("none") or show ("block") elements.
- Code: `modal.style.display = "none";`

DOB: EventListener

- ❖ An Event Listener is a procedure in JavaScript that **"listens"** or waits for a specific event (like a click or keypress) to occur on a specific HTML element. When that event happens, it executes a defined function (callback).
- ❖ Common Types of Event Listeners
 - Mouse Events: click, mouseover, mouseout, dblclick
 - Keyboard Events: keydown, keyup
 - Form Events: submit, input
 - Window/Document Events: load, scroll
- ❖ Important Concepts
 - The Syntax Rule (No Parentheses!)
 - When adding a listener, you pass the name of the function, not the execution.
 - Correct: `btn.addEventListener("click", myFunction);` (Run later).
 - Wrong: `btn.addEventListener("click", myFunction());` (Run now).

DOB: EventListener

❖ Important Concepts

➤ Preventing Default Behavior (e.preventDefault)

- Browsers have built-in behaviors (e.g., clicking a "Submit" button refreshes the page). You often need to stop this.
- Usage: Essential for Form Handling and Single Page Applications.\

➤ The Event Object (e)

- JavaScript automatically passes a hidden argument containing details about the event.
- **e.target**: Returns the specific HTML element that was clicked.
- **e.key**: Returns the specific key pressed (e.g., "Enter").

Local Storage

- ❖ A storage mechanism that allows JavaScript sites and apps to save key/value pairs in a web browser with no expiration date.
- ❖ Permanent. The data stays there even if the user refreshes the page, closes the tab, or restarts the browser. It only disappears if the user manually clears their browser cache or the code specifically deletes it.
- ❖ Shared across all tabs and windows of the same browser for that specific website (domain).
- ❖ Capacity: Generally around 5MB - 10MB (huge compared to cookies).
- ❖ Why
 - User Preferences: Saving "Dark Mode" settings or font size choices.
 - Shopping Carts: Remembering items in a cart so they are still there when the user comes back 2 days later.
 - Authentication: Storing a "Keep me logged in" token (though secure cookies are often better for this).

Local Storage

// Saving a preference

```
localStorage.setItem("theme", "dark");
```

// Even if I restart the computer, this code works:

```
let currentTheme = localStorage.getItem("theme"); // Returns "dark"
```

Session Storage

- ❖ **A storage mechanism** that **saves data** only for the **duration of the session**.
- ❖ **Temporary.** The data is valid only as long as the Tab is open. The moment the user closes the tab or the browser, the data is wiped instantly.
- ❖ **Isolated. It is not shared between tabs.** If you open google.com in Tab A and google.com in Tab B, they have completely separate Session Storages.
- ❖ **Why?**
 - Sensitive Data: Banking login sessions (you want this to disappear when the user leaves).
 - If a user fills out Page 1 of a form and clicks "Next," you can save their answers in Session Storage so they don't lose them if they hit "Back."
 - Single-Session Settings: Filtering a list of products (e.g., "Sort by Price") that should reset next time they visit.

Session Storage

// Saving a temporary status

```
sessionStorage.setItem("isLoggedIn", "true");
```

// If I close the tab and reopen it:

```
let status = sessionStorage.getItem("isLoggedIn"); // Returns null (It's gone)
```

Graphical User Interface

- ❖ A Graphical User Interface (GUI) enables users to interact with digital devices through graphical elements like icons and buttons.
- ❖ Hover Effects:
 - Instead of just changing colors (like CSS), JavaScript allows for complex logic when the mouse moves over an element.
 - The Logic: You listen for two events: mouseover (mouse enters) and mouseout (mouse leaves).
 - Use Case: Showing a "Tooltip" (a small hint box) that follows the mouse, or triggering an animation on a completely different part of the page.

Components of a Graphical User Interface



Graphical User Interface

- ❖ **Dynamic Menus (The "Hamburger" Menu):**
 - This creates navigation bars that slide in or drop down on demand, essential for mobile views.
 - The Logic: You create a button (the "trigger"). When clicked, JavaScript toggles a CSS class (like `.active` or `.show`) on the menu container.
- ❖ **Modals (Popups):**
 - A Modal is a window that forces the user to interact with it before returning to the main page.
 - The Structure: It usually has two parts:
 - **Overlay:** A dark, semi-transparent background that covers the whole screen.
 - **Content Box:** The white box floating on top (centered).

Components of a Graphical User Interface



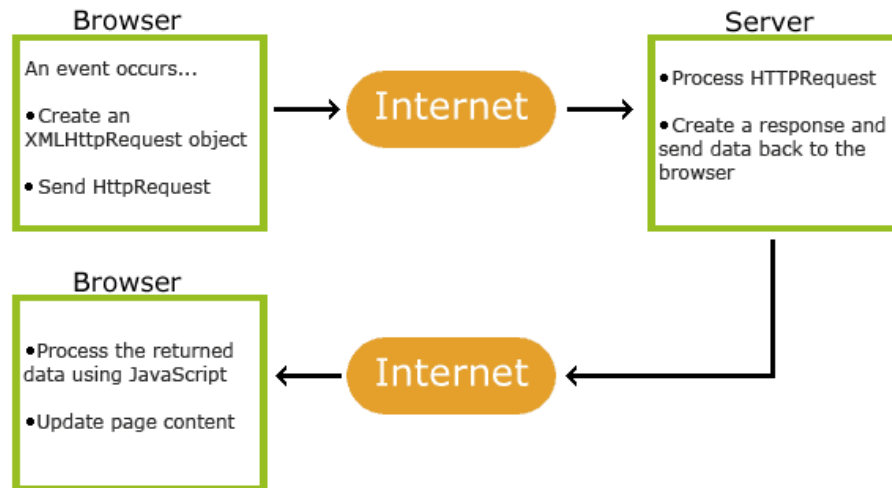
JavaScript Library: jQuery

- ❖ jQuery is a fast, small, and feature-rich JavaScript Library.
- ❖ It is designed to simplify HTML document traversal and manipulation, event handling, animation, and Ajax interactions for rapid web development.
- ❖ Its motto is "Write Less, Do More."
- ❖ **The Magic Selector (\$) - DOM Shortcuts**
 - In standard JavaScript, selecting elements can be wordy. jQuery solves this with the \$ (Dollar Sign) function.
 - The \$ sign creates a jQuery Object that wraps the HTML element, giving it special powers.
 - Syntax: \$("selector")
 - Examples:
 - Select by ID: \$("#myBtn") (Instead of document.getElementById("myBtn"))
 - Select by Class: \$(".error")
 - Select by Tag: \$("p")

JavaScript Library: jQuery

❖ AJAX Basics (No Refresh)

- AJAX stands for Asynchronous JavaScript And XML.
 - It allows the webpage to talk to the server (get data) without reloading the page.
 - jQuery's Role: Writing AJAX in raw JavaScript is difficult (requires 10+ lines). jQuery does it in 1 line.



Modern JavaScript (ES6+)

❖ Let and Const (Variable Declaration):

- Concept: Replacing var with smarter, safer variables.
- Formal Definition: let declares a block-scoped variable that can be reassigned. const declares a block-scoped variable that cannot be reassigned (it is constant).
- They observe Block Scope.
- This means if you define them inside an if block or loop { ... }, they do not exist outside of it. var would leak out; these do not.

```
// --- 1. Block Scope Logic ---
if (true) {
    var oldVar = "I survive outside!";
    let newLet = "I die here!";
}
console.log(oldVar); // Works
// console.log(newLet); // Error: newLet

// --- 2. Reassignment Logic ---
let score = 10;
score = 20; // ✅ Allowed

const pi = 3.14;
// pi = 3.14159; // ❌ Error: Assignment
```

Modern JavaScript (ES6+)

❖ Arrow Functions (=>):

- An Arrow Function is a compact alternative to a traditional function expression that does not have its own bindings to this or super.
- If the function has only one line of code, you can remove the {} and the return keyword. This is called an Implicit Return.

```
// 1. Old Way (ES5)
```

```
const add = function(a, b) {  
    return a + b;  
};
```

```
// 2. Modern Way (ES6 Arrow)
```

```
// Removed 'function' keyword
```

```
const addModern = (a, b) => {  
    return a + b;  
};
```

```
// 3. The "One-Liner" (Professional Style)
```

```
// Removed braces {} and 'return' keyword
```

```
const addPro = (a, b) => a + b;
```

Modern JavaScript (ES6+)

❖ Destructuring

- Destructuring assignment is a syntax that allows you to "unpack" values from arrays, or properties from objects, into distinct variables.
- It saves you from writing repetitive code

```
const user = { name: "Alex", age: 25, country: "Nepal" };

// Old Way:
// let name = user.name;
// let age = user.age;

// Modern Way:
const { name, age } = user; // Creates variables 'name' and 'age'
console.log(name); // "Alex"
```

```
const colors = ["Red", "Green", "Blue"];

// Extract first two colors
const [first, second] = colors;
console.log(first); // "Red"
```

Array Destructuring

Object Destructuring

Spread and Rest Operators (...)

❖ Spread Operator (Expands)

- Expands an array into individual elements.
- Combining arrays or passing array values into a function.

❖ Rest Parameter (Collects)

- Collects multiple elements and condenses them into a single array.
- When a function needs to accept an unlimited number of arguments.

```
const fruits = ["Apple", "Banana"];
const veggies = ["Carrot", "Potato"];

// Combine them easily
const food = [...fruits, ...veggies];
console.log(food); // ["Apple", "Banana",
```

```
// The function doesn't know how many numbers are coming
function sumAll(...numbers) {
  // 'numbers' is now an array: [10, 20, 30]
  return numbers.reduce((total, num) => total + num);
}

console.log(sumAll(10, 20, 30)); // 60
```

Why Asynchronous JavaScript?

- ❖ In standard programming (Synchronous), line 2 waits for line 1 to finish.
- ❖ But in Web Development, some tasks take time (like downloading a file or fetching data).
- ❖ We cannot freeze the whole website while waiting.
- ❖ Synchronous vs. Asynchronous
 - **Synchronous:** In synchronous code, the browser reads your code from top to bottom. If line 2 takes 5 seconds, Line 3 cannot run until Line 2 is finished.
 - **Asynchronous:** , JavaScript offloads heavy tasks (like downloading a file or a timer) to the browser's background APIs and keeps running the rest of the code.

Note: If you write a Synchronous API call (which is possible but bad practice), your website will crash/freeze for the user every time they click a button. You must use Async (Promises/Async-Await) for anything that takes time.

The Evolution of Async Code

❖ Stage 1: Callbacks (The Old Way)

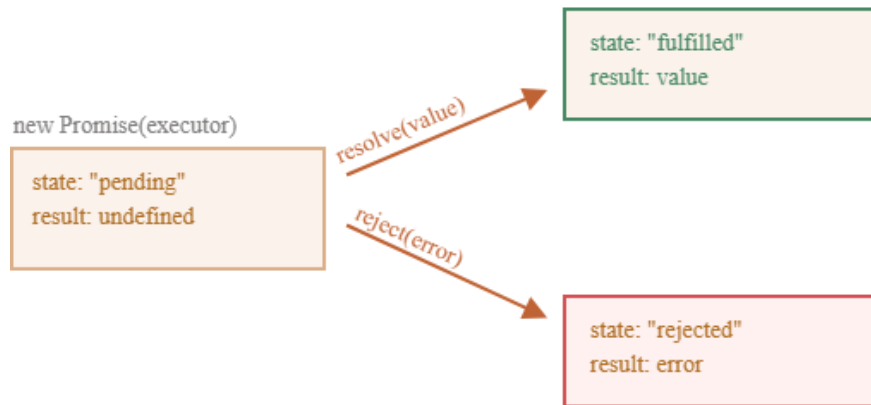
- Passing a function into another function to run "when you are done."
- A function passed as an argument to another function.
- If you have to do 3 things in a row (Login -> Get Data -> Save Data), you end up nesting callbacks inside callbacks. This is called "**Callback Hell**".

```
1 asyncOperation1(function(result1){~  
2   ..asyncOperation2(result1, function(result2){~  
3     ....asyncOperation3(result2, function(result3){~  
4       .....// More nested callbacks ... ~  
5     ....});~  
6   ..});~  
7 });~  
8 };
```

The Evolution of Async Code

❖ Stage 2: Promises (The Improvement)

- An object that represents a future event.
- A Promise is an object that represents the eventual completion (or failure) of an asynchronous operation.
- 3 States:
 - Pending: Working on it...
 - Resolved (Fulfilled): Success!
 - Rejected: Failed!



The Evolution of Async Code

❖ Stage 3: Async / Await (The Modern Standard)

- It makes async code look like normal synchronous code.
 - **async:** Put this in front of a function to force it to return a Promise.
 - **await:** Put this in front of a promise to pause the code until the promise is resolved.